Assignment 2: Breaking AES through DPA

Shantanu Mukhopadhyay [200541793]

Seeptember 26, 2024

1 Problem 1: Analyze power traces

Analysis on '*traces.csv*' power trace file

Figure 1: Plot the first power trace, corresponding to the first encryption operation.

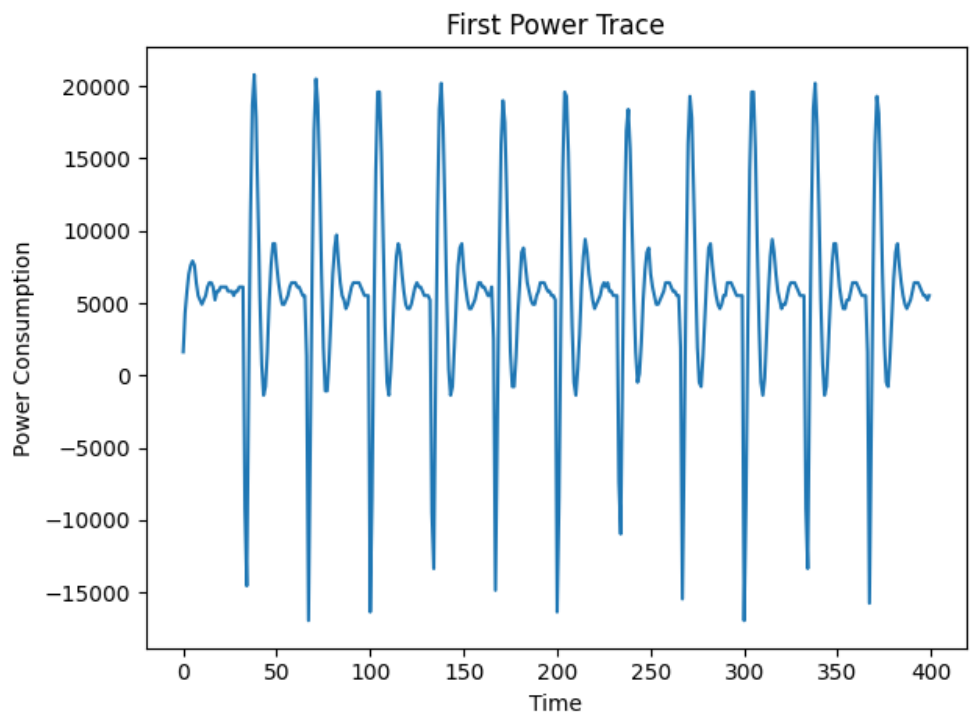

Figure 1: First Power Trace

Question 1: Analyze the power trace. What are the peaks in the trace? How is AES implemented?

Answer 1: The first power trace corresponding to the first encryption operation has a total of 22 peaks(on either side positive and negative), these peaks correspond to the 11 rounds of operations of:

1. **Initial transformation:**: only AddRoundKey is done here.
2. **Next 9 rounds of AES**: all subroutines are performed:SubBytes, AddRoundKey, ShiftRows, and MixColumns
3. **Final transformation**: All Subroutines except the MixColumns are performed.

The AES-128 encryption consists of 10 rounds, each with the following subroutines:

1. **AddRoundKey**: The state is XOR'd with the corresponding round key.
2. **SubBytes**: Each byte in the state matrix is substituted with a value from the S-box.
3. **ShiftRows**: Rows of the state matrix are left-shifted by different offsets.

4. **MixColumns**: Columns of the state matrix are mixed using a fixed matrix in a Galois field (except in the final round).

Hence, if there is a change in any of these parameters or it's values, it will propagate to the final output and will vary the final encrypted value change by a lot. Note: The first trace was pre-processed and shifted down to match with all other trace values to make our DPA attack reliable.

2 Problem 2: Perform the DPA Attack

Question 2: What is a good power model for the target operation? Justify it.
Answer 2: I have chosen the Hamming weight model as a good power model for the DPA attack. The Hamming weight model estimates the power consumption based on the number of '1' bits in a byte. This model is effective because the power consumption of digital circuits often correlates with the number of bits that switch from '0' to '1' or vice versa.

In the hypothesis table, the mapped input byte and key transition give us an estimation of power level changes. Using the Hamming weight model, we can directly estimate these changes and apply linear correlation tests, such as Pearson's correlation, to identify the key with the highest correlation to our power model. This approach is practical and widely used in side-channel analysis due to its simplicity and effectiveness in capturing power consumption patterns.

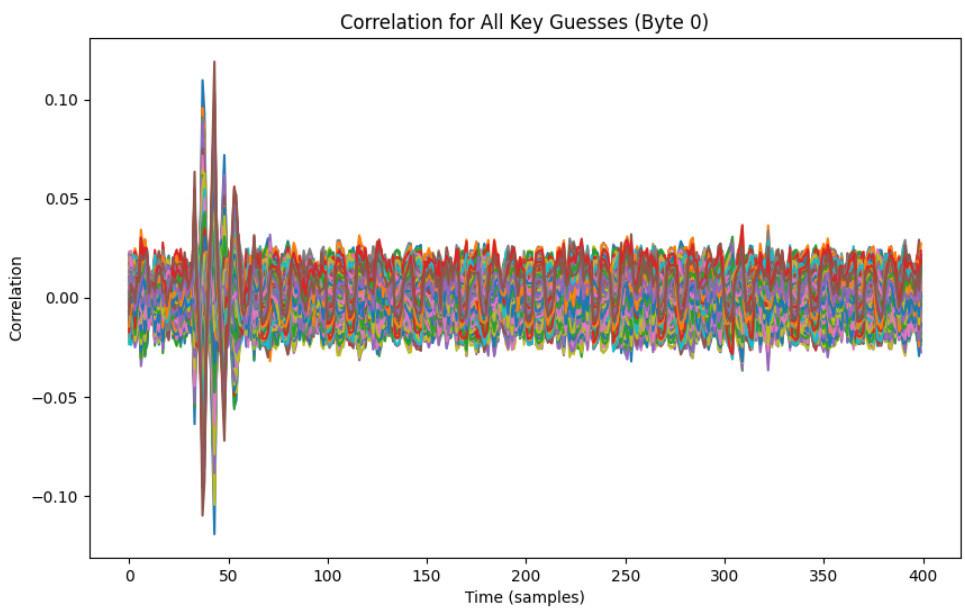

Figure 2: All Key Guesses for the first byte of the Key

Question 3: Which one is more likely to be the correct key guess? Why do you specifically see these two results?
Answer 3: The key value corresponding to the highest absolute peak value is the most likely correct key guess. This is because the highest peak indicates the strongest linear correlation between the key guess and the observed power traces, suggesting that this key guess is most closely related to the actual power consumption changes during the AES-ECB execution. The second result also

shows a high correlation, making it the second-best guess for the key(positive and negative peaks here).

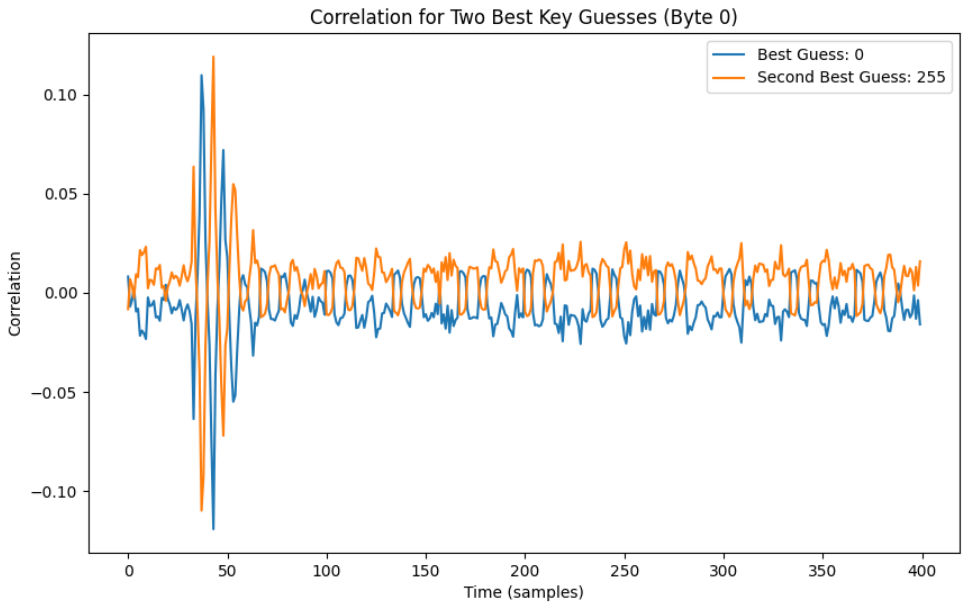

Figure 3: Plot for maximum correlation for best 2 key guesses for first byte(MSB)

Question 4: What is the maximum leak point (in time domain)? Plot the evolution" of all key hypothesis for 10000 measurements at this particular time instance.
Answer 4: Looking at the correlation for the key guesses, we can see that the maximum leak point i.e. where the hardware leaks power for the corresponding AES-ECB operation was at the **43rd sample** in the time domain.

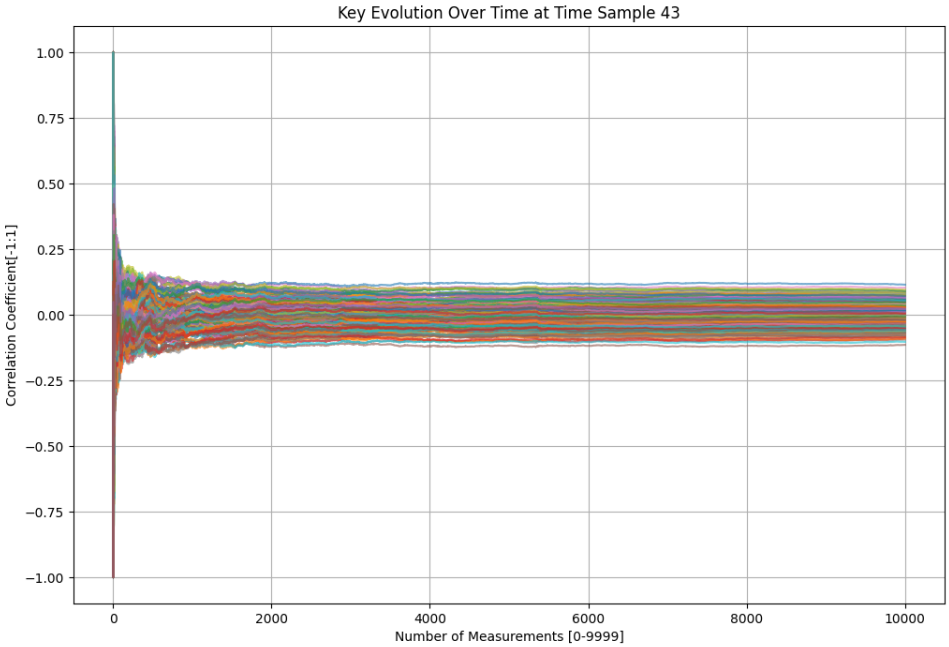

Figure 4: Evolution of all key hypothesis for 10000 measurements at 43 time sample

Question 5: Starting at how many traces, does the key guess with maximum correlation show highest correlation?
Answer5: When scale is zoomed-in the key guess with maximum correlation in the figure 4 (blue consistent line) shows its maximum value at nearly around 30 samples(indicated by blue arrow pointing the peak). As you can see in here:

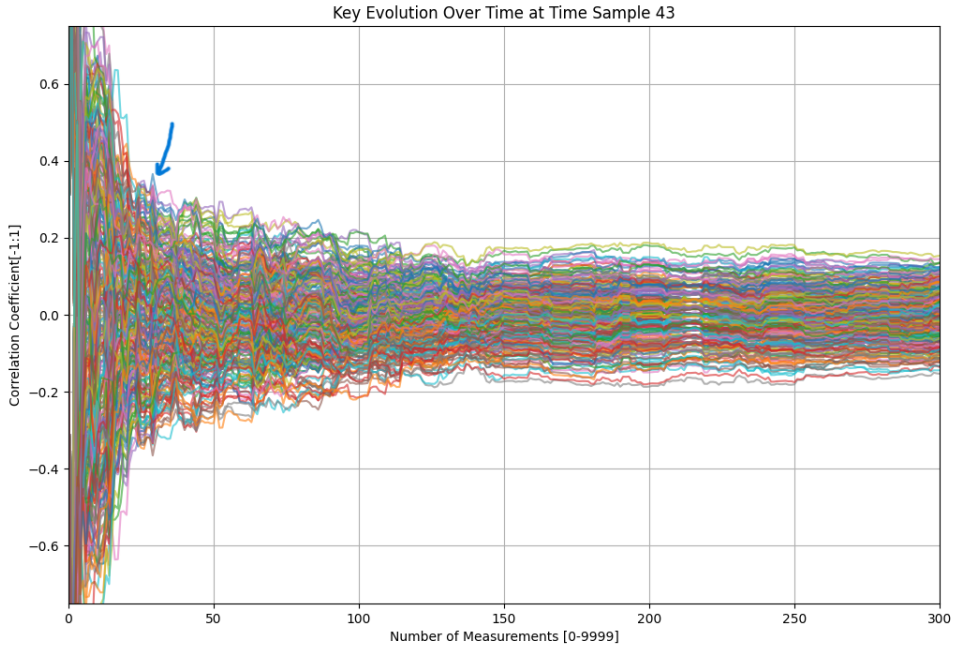

Figure 5: Zoomed-in key evolution plot

Question 6: Apply the DPA on the entire key. What is the 128-bit AES secret key?
Answer6: The final 128_bit AES Secret key is '**0x000102030405060708090a0b0c0d0e0f**'. My code gives me this answer:000102fcfb7af9f80809f5f40cb20e0f, which did not work for the AES encryption. I had printed top 4 guesses with their correlation value and this issue was seen that both peaks were of same values hence there was conflict in mapping in guesses, I had printed top 4 guesses for all bytes.Then mapped over the values of the power trace values and noted down the pattern and brute forcing the values in the AES encryption for the given input plaintext I got the correct corresponding ciphertext. Hence, I was able to figure out the final AES key using very small calculations instead of brute-forcing.

3 Problem 3: Evaluating DPA

Question 7: What is the mean time to disclosure, i.e. average number of traces required to extract the key for your DPA attack?
Answer 7: The mean time to disclosure i.e the average number of traces required to extract the key for DPA attack was **240**. This was found by calculating the peak in the key evolution graphs and averaging over 16 bytes.
Question 8: Compare this with the theoretical cryptanalysis (i.e., brute-force attack) of AES, what is the reduction ratio in the number of traces required to break AES? Why is DPA so powerful?
Answer 8: AES-128 uses a 128-bit key, yielding 2^{128} possible key combinations, making brute-force decryption computationally infeasible. A brute-force attack requires testing all possible keys, which would take millions of years even with modern hardware. In contrast, Differential Power Analysis (DPA) leverages power consumption data during encryption to reveal parts of the key byte by byte. This reduces the complexity from 2^{128} to 2^8 (256 possibilities per byte). With DPA, the

mean time to disclosure was 240 traces, achieving a massive reduction compared to brute-force, as DPA efficiently correlates power traces with key hypotheses. The mean time to disclosure obtained was 240 for this DPA attack, hence reduction ratio is essentially comparing the effort required for DPA v/s brute-force attacks and this would be a massive reduction compared to brute-forcing AES-128, computed as:

$$\text{Reduction Ratio} = \frac{\text{Number of trials in Brute-force}}{\text{Number of Traces in DPA}} \tag{1}$$

$$\text{Reduction Ratio} = \frac{2^{128}}{240} \approx \frac{3.4 \times 10^{38}}{240} \approx 1.416 \times 10^{36} \tag{2}$$

This shows that the **number of traces required to break AES using DPA is reduced by a factor of approximately** 10^{36} **, compared to brute-force attacks**. Thus, DPA attacks are very strong compared to brute-force attacks as :

1. *Power Leakage:* AES is theoretically secure against brute-force attacks due to its large key size, but DPA exploits the fact that physical devices (like CPUs or hardware modules) leak information through power consumption. These power traces carry side-channel information that can be correlated to the key being used.

2. *Byte-by-Byte Attack:* Instead of attacking the full 128-bit key at once, DPA attacks only a portion at a time. This reduces the attack complexity by a huge factor as seen above.

3. *Relatively Few Traces*: As demonstrated in your analysis, a DPA attack can succeed with only a few hundred or thousand traces, which is exponentially fewer than the 2^{128} combinations in brute-force attacks.

Question 9: How much time it took you to perform the experiment and complete questions 1-8? **Answer 9:** The average time taken to complete experiment and questions 1-8 was **15 hours**.

4 Extending the DPA Attack

Now use the '*traces2.csv*' and implement the DPA

Question 10: Why is it failing now? What is the difference in hardware implementation causing this and how would you change the attack to break the implementation?

Answer 10: While changing our trace file to traces2, and after performing DPA attack there were **no** significant stand-alone peaks indicating linearity of the keys to the power trace.

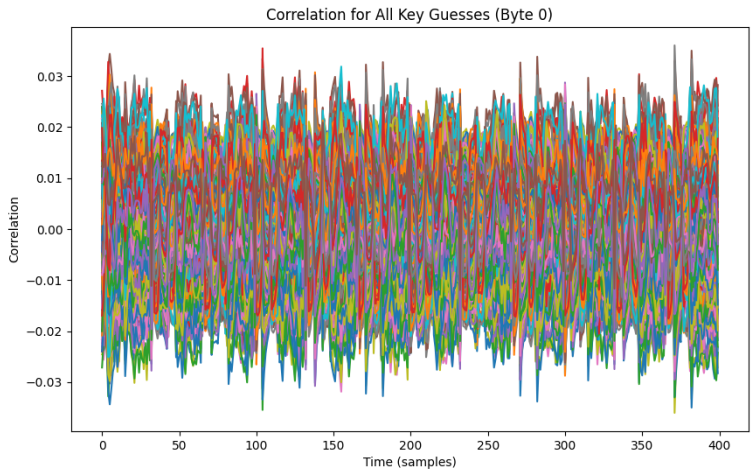

Figure 6: All Key Guesses for the first byte of the Key for traces2.csv

This is failing because the power model shows no significant single-point linear correlation with the power trace, since there could be a set of **DPA attack countermeasures** implemented in the design, such as shielding, specialized circuitry to minimize power variations, masking techniques or noise injection, or randomized power consumption techniques in the design during AES operations.

To break these implementations we could try different techniques such as:

1. *Use of other side-channel attacks:* We can explore other side-channel attacks such as electromagnetic(EM) analysis, timing analysis, maybe through fault injection, etc.

2. *Higher Order DPA:* If the design uses masking techniques, we can consider applying higher-order DPA (e.g., second-order DPA), which analyzes the power traces for more complex correlations. This can help to reveal information even with masking.

3. *Template Attacks:* Use template attacks, where you create a model based on the expected power consumption of the device for different key hypotheses.

Question 11: Assume that you do not have access to input plaintext but only to output ciphertext (i.e. ciphertext-only cryptanalysis), how would you modify the attack?

Answer 11: Ciphertext-Only Analysis (COA): In this type of attack, only some ciphertext is known, and the attacker tries to find the corresponding encryption key and plaintext. It is the hardest to implement, however, if the encryption algorithm has weaknesses or the attacker can exploit certain patterns in the ciphertexts, it is still possible to recover the plaintext or key. The attack can be modified to work on ciphertexts only by:

1. *Statistical Analysis*: Analyze the frequency of patterns in the ciphertext, especially in weaker encryption schemes, to detect potential correlations with common plaintext patterns.

2. *Look for Repeated Blocks:* In block ciphers, modes like ECB can reveal repeated patterns in the ciphertext, corresponding to repeated blocks of plaintext.

3. *Exploit Known Structures:* If the plaintext follows a known format (e.g., file headers or common language structures), use that knowledge to infer parts of the plaintext.This technique helps in narrowing down possible key values.

4. *Dictionary/Known Ciphertext Attacks:* Use pre-computed ciphertext-plaintext pairs, if available, to match ciphertexts to known plaintexts.

5. *Brute Force:* As a last resort, attempt to brute-force the key if the key length is short or if the encryption scheme is weak.